# Phase 5: Designing Hash Functions and Choosing a Method — Full Structured Note

## 1. Current Progress Analysis

Before Phase 5, you already learned the main hashing path:

| Phase | What you learned |
|---|---|
| **Phase 1** | Hashing basics: key, index, hash table, hash function, modulus hashing, and collisions. |
| **Phase 2** | **Open hashing / chaining**: collisions are stored in linked lists. |
| **Phase 3** | **Closed hashing / linear probing**: collisions are handled inside the same array by moving one slot at a time. |
| **Phase 4** | **Quadratic probing and double hashing**: better probing methods that reduce clustering. |

Now Phase 5 teaches the final skill:

> How to design a good hash function and how to choose the right hashing method.

This phase is important because a collision method cannot save a very bad hash function. If the hash function sends too many keys to the same place, the hash table becomes slow.

Simple idea:

```
Good hash function + good table size + suitable collision method = efficient hashing
```

---

## 2. Main Goal of Phase 5

The goal of Phase 5 is to understand how to make hashing work well in practice.

In this phase, you learn:

1. Hash function design goals.
2. Uniform distribution.
3. Deterministic output.

4. Why you must know your data before choosing a hash function.
5. Table-size rules for chaining and probing.
6. Why prime table sizes are useful.
7. Modulus hashing.
8. Mid-square hashing.
9. Folding hashing.
10. String hashing using ASCII values.
11. Built-in hashing in modern languages.
12. How to compare chaining, linear probing, quadratic probing, and double hashing.
13. A complete C++ hash-function lab program.
14. Common beginner mistakes.
15. A final checklist for choosing a method.

---

## 3. What Is a Hash Function?

A hash function is a formula that converts a key into an index.

General process:

```
Key → Hash Function → Index
```

Example:

```
Key = 25
Hash function = key % 10
25 % 10 = 5
```

So key `25` maps to index `5`.

Simple meaning:

> The hash function tells the program where to store or search for a key.

---

## 4. Why Hash Function Design Matters

In earlier phases, you learned what to do when a collision happens.

Phase 5 focuses on reducing bad collisions before they happen.

Example of a bad hash function:

```
Keys: 5, 35, 95, 145, 175
Hash function: H(x) = x % 10
```

Calculations:

```
5 % 10 = 5
35 % 10 = 5
95 % 10 = 5
145 % 10 = 5
175 % 10 = 5
```

All keys go to index `5`.

This is bad because only one bucket is used, while the rest of the table stays empty.

Simple lesson:

> A good hash function spreads keys. A bad hash function creates unnecessary collisions.

---

## 5. Goal 1: Uniform Distribution

Uniform distribution means keys are spread evenly across the hash table.

### Good Distribution

```
Index 0: 1 key
Index 1: 2 keys
Index 2: 1 key
Index 3: 2 keys
Index 4: 1 key
```

This is good because no one place becomes too crowded.

### Bad Distribution

```
Index 0: empty
Index 1: empty
Index 2: empty
Index 3: empty
Index 4: 7 keys
```

This is bad because the table starts acting like a slow list.

In chaining, bad distribution creates long chains.

In probing, bad distribution creates long probe sequences or clusters.

Simple meaning:

```
Uniform distribution = spread the keys across the table
```

---

## 6. Goal 2: Deterministic Output

A hash function must be deterministic.

Deterministic means:

```
Same key → Same index every time
```

Example:

```
H(25) = 5
```

If key `25` is inserted using index `5`, then searching for key `25` must also calculate index `5`.

If the same key gives different indexes at different times, the search will look in the wrong place.

Simple rule:

> A hash function must not behave randomly for the same key.

---

## 7. Know Your Data Before Choosing a Hash Function

Hash function design depends on the pattern of the keys.

A hash function that works well for one data set may be poor for another.

### Example: All Keys End in 5

Keys:

```
5, 35, 95, 145, 175
```

Using:

```
H(x) = x % 10
```

All keys map to index `5`.

| Key | Calculation | Index |
|---|---|---|
| 5 | 5 % 10 | 5 |
| 35 | 35 % 10 | 5 |
| 95 | 95 % 10 | 5 |
| 145 | 145 % 10 | 5 |
| 175 | 175 % 10 | 5 |

This is bad.

### Better Idea

Use another part of the number:

```
H(x) = (x / 10) % 10
```

This uses the tens digit instead of the last digit.

| Key | x / 10 | (x / 10) % 10 | Index |
|---|---|---|---|
| 35 | 3 | 3 % 10 | 3 |
| 95 | 9 | 9 % 10 | 9 |
| 145 | 14 | 14 % 10 | 4 |
| 175 | 17 | 17 % 10 | 7 |

This spreads the keys better.

Main lesson:

> First inspect the key pattern. Then choose the hash function.

---

## 8. Table Size Guidance

The table size affects how well hashing works.

A poor table size can increase collisions.

A better table size can reduce regular collision patterns.

| Collision method | Table-size / loading guidance |
|---|---|
| **Chaining** | Flexible. Loading factor can be greater than `1` because chains can grow. |
| **Linear probing** | Use about double the number of keys. Keep `lambda <= 0.5`. |
| **Quadratic probing** | Keep the table not too full. Usually also use `lambda <= 0.5`. |
| **Double hashing** | Needs a good table size and a good second hash function. |
| **Modulus hashing** | Prime table sizes such as `11`, `13`, and `17` often help. |

---

## 9. Loading Factor Reminder

The loading factor tells us how full the hash table is.

Formula:

```
lambda = n / size
```

Where:

| Symbol | Meaning |
|---|---|
| `lambda` | Loading factor |
| `n` | Number of stored keys |
| `size` | Number of table slots or buckets |

Example:

```
n = 5
size = 10
lambda = 5 / 10 = 0.5
```

This means the table is half full.

### Loading Factor in Chaining

In chaining, the loading factor can be greater than `1` because each bucket can hold a linked list.

Example:

```
n = 20
size = 10
lambda = 20 / 10 = 2
```

This means the average chain length is about `2`, if the keys are spread evenly.

### Loading Factor in Probing

In closed hashing methods, empty slots are very important.

For linear probing, the rule is:

```
lambda <= 0.5
```

Simple meaning:

```
Keep the table at most half full.
```

If you expect `50` keys, use about `100` slots.

---

## 10. Prime Table Size Tip

For modulus hashing, prime table sizes often reduce regular patterns.

Examples of prime sizes:

```
11, 13, 17, 19, 23
```

Why primes help:

Many keys have patterns. For example, they may end in the same digit, or they may be multiples of a common number.

A prime table size can help avoid some regular collision patterns.

Simple rule:

> For modulus hashing, prefer a prime table size when possible.

---

# Part A: Hash Function Types

---

## 11. Modulus Hashing

Modulus hashing uses the remainder after division.

Formula:

```
H(x) = x % size
```

C++ idea:

```cpp
int hashModulus(int key, int size) {
    return key % size;
}
```

### Example with Size 10

```
15 % 10 = 5
```

So key `15` maps to index `5`.

### Example with Size 13

```
15 % 13 = 2
```

So key `15` maps to index `2`.

### Advantages

Modulus hashing is:

```
Simple
Fast
Easy to code
Commonly used
```

### Weakness

It can create many collisions if the key pattern matches the table size badly.

Example:

```
Keys ending in 5 with table size 10
```

All may map to index `5`.

---

## 12. Mid-Square Hashing

Mid-square hashing uses the middle digit or middle digits of the squared key.

Steps:

```
1. Square the key.
2. Take the middle digit or middle digits.
3. Use that value as the index.
4. If needed, apply modulus to fit the table size.
```

Why square the key?

Squaring mixes the digits. This can be better than using only the last digit.

### Dry Run 1: Key 11

```
Key = 11
11² = 121
Middle digit = 2
Index = 2
```

### Dry Run 2: Key 13

```
Key = 13
13² = 169
Middle digit = 6
Index = 6
```

| Key | Square | Middle digit | Index |
|---|---|---|---|
| 11 | 121 | 2 | 2 |
| 13 | 169 | 6 | 6 |

If the square is long, take the middle two or three digits.

If the number is still too large, apply modulus.

---

## 13. Folding Hashing

Folding hashing is useful for large keys such as phone numbers, ID numbers, or account numbers.

Basic idea:

```
Break the large key into smaller pieces, add the pieces, then use the result.
```

### Steps

```
1. Break the key into equal-sized pieces.
2. Add the pieces.
3. Optionally fold again.
4. Apply modulus if the table is smaller.
```

### Dry Run: 123347

Key:

```
123347
```

Break into pieces:

```
12, 33, 47
```

Add:

```
12 + 33 + 47 = 92
```

Fold again:

```
9 + 2 = 11
```

So the result is:

```
Index = 11
```

If the table size is `10`, apply modulus:

```
11 % 10 = 1
```

So the final index would be `1`.

---

## 14. String Hashing

Strings cannot directly be used as array indexes.

A string must first be converted into a number.

A simple beginner method is to add ASCII values.

### ASCII Values

| Character | ASCII value |
|---|---|
| `A` | 65 |
| `B` | 66 |
| `C` | 67 |

### Dry Run: "ABC"

String:

```
ABC
```

ASCII values:

```
A = 65
B = 66
C = 67
```

Add them:

```
65 + 66 + 67 = 198
```

Apply modulus with table size `13`:

```
198 % 13 = 3
```

So:

```
"ABC" maps to index 3
```

### Weakness of Simple ASCII Sum

The ASCII-sum method is simple, but it can create collisions.

Example:

```
"ABC" and "CBA"
```

Both have the same characters, so both have the same sum.

Real string hashing usually uses stronger formulas, but ASCII sum is good for understanding the beginner idea.

---

## 15. Built-In Hashing

Modern programming languages often provide built-in hashing.

| Language | Example hashing feature |
|---|---|
| Java | `hashCode()` |
| C++ | `unordered_map`, `unordered_set` |
| Python | `dict`, `set` |
| .NET / C# | hash-code mechanisms |

Even when a language provides hashing, you still need to understand:

```
Collisions
Table size
Loading factor
Uniform distribution
Average-case O(1)
Worst-case slowdown
```

Simple lesson:

> Built-in hashing is useful, but understanding hashing helps you know why performance is good or bad.

---

# Part B: Choosing the Right Collision Method

---

## 16. Collision Reminder

A collision happens when two or more keys map to the same index.

Example:

```
15 % 10 = 5
25 % 10 = 5
```

Both keys want index `5`.

The collision method decides what to do next.

---

## 17. Chaining

In chaining, each table index points to a linked list.

Example:

```
Index 5: 15 -> 25 -> 35 -> NULL
```

### Best Use Case

Use chaining when:

```
Extra memory is allowed.
Deletion matters.
The number of keys may grow.
```

### Strength

Deletion is simple because you remove a node from a linked list.

### Weakness

It uses extra memory.

If the hash function is bad, one chain can become very long.

---

## 18. Linear Probing

In linear probing, all keys stay inside the array.

If the slot is full, move to the next slot.

```
Try index 5
If full, try index 6
If full, try index 7
```

### Best Use Case

Use linear probing when:

```
You want a simple array-only method.
The table will stay sparse.
You can keep lambda <= 0.5.
```

### Strength

It is simple and uses no linked lists.

### Weakness

It creates primary clustering.

Deletion is difficult because clearing a slot can break future searches.

---

## 19. Quadratic Probing

Quadratic probing uses square jumps.

Formula:

```
H'(x) = (H(x) + i²) % size
```

Jump pattern:

```
0, 1, 4, 9, 16, ...
```

### Best Use Case

Use quadratic probing when:

```
You want less clustering than linear probing.
You still want everything inside one array.
```

### Strength

It spreads keys better than linear probing.

### Weakness

It still depends on table size and loading factor.

Keys with the same starting index can still follow the same square-jump pattern.

---

## 20. Double Hashing

Double hashing uses two hash functions.

```
H1(x) = starting index
H2(x) = jump size
```

Formula:

```
H'(x) = (H1(x) + i * H2(x)) % size
```

### Best Use Case

Use double hashing when:

```
You want strong collision spreading.
You want different keys to follow different probe paths.
You can design a good H2(x).
```

### Strength

It usually reduces clustering better than linear probing and quadratic probing.

### Weakness

It is more complex and needs a good second hash function.

Important rule:

```
H2(x) must never be 0
```

If `H2(x) = 0`, the probe will not move.

---

## 21. Method Comparison Table

| Method | Best beginner use case | Strength | Weakness |
|---|---|---|---|
| **Chaining** | Extra memory is allowed and deletion matters | Simple deletion; can handle `lambda > 1` | Uses linked lists and extra memory |
| **Linear probing** | Keep everything inside one sparse array | Simple array implementation | Primary clustering and deletion issues |
| **Quadratic probing** | Reduce clustering compared with linear probing | Square jumps spread keys more | Table-size issues can still happen |
| **Double hashing** | Break collision patterns strongly | Key-specific jumps reduce clustering | Needs a good second hash function |

Clustering reduction ranking:

```
Linear probing < Quadratic probing < Double hashing
```

---

# Part C: Complete C++ Hash Function Lab Program

---

## 22. Complete Program

This program demonstrates modulus hashing, mid-square hashing, folding hashing, and string hashing.

```cpp
#include <iostream>
#include <string>
using namespace std;

int hashModulus(int key, int size) {
    return key % size;
}

int middleDigit(int number) {
    string s = to_string(number);
    int mid = s.length() / 2;
    return s[mid] - '0';
}

int hashMidSquare(int key, int size) {
```

```
    int square = key * key;
    int mid = middleDigit(square);
    return mid % size;
}

int hashFolding(int key, int size) {
    int sum = 0;

    while (key > 0) {
        sum += key % 100;   // take last two digits
        key /= 100;         // remove last two digits
    }

    return sum % size;
}

int hashStringSum(const string& key, int size) {
    int sum = 0;

    for (char ch : key) {
        sum += static_cast<int>(ch);
    }

    return sum % size;
}

int main() {
    int size = 13; // prime table size

    cout << "Modulus: 15 -> " << hashModulus(15, size) << endl;
    cout << "Mid-square: 11 -> " << hashMidSquare(11, size) << endl;
    cout << "Mid-square: 13 -> " << hashMidSquare(13, size) << endl;
    cout << "Folding: 123347 -> " << hashFolding(123347, size) << endl;
    cout << "String sum: ABC -> " << hashStringSum("ABC", size) << endl;

    return 0;
}
```

---

## 23. Program Explanation

`hashModulus`

```
return key % size;
```

This returns the remainder after division.

Example:

```
15 % 13 = 2
```

### `middleDigit`

This converts the number to a string and returns the middle digit.

Example:

```
121 → middle digit 2
```

### `hashMidSquare`

This squares the key, takes the middle digit, and applies modulus.

Example:

```
13² = 169
Middle digit = 6
6 % 13 = 6
```

### `hashFolding`

This takes two-digit pieces from the key and adds them.

For `123347`:

```
47 + 33 + 12 = 92
92 % 13 = 1
```

Note: The curriculum dry run also shows folding again:

```
12 + 33 + 47 = 92
9 + 2 = 11
```

`hashStringSum`

This adds the ASCII values of the string characters.

For `ABC`:

```
A = 65
B = 66
C = 67
65 + 66 + 67 = 198
198 % 13 = 3
```

---

## 24. Expected Program Output

Using table size `13`, the output is:

```
Modulus: 15 -> 2
Mid-square: 11 -> 2
Mid-square: 13 -> 6
Folding: 123347 -> 1
String sum: ABC -> 3
```

---

## 25. Complexity Notes

Hash calculation is usually fast.

| Hash function | Typical cost | Why |
|---|---|---|
| Modulus hashing | `O(1)` | One remainder calculation |
| Mid-square hashing | `O(1)` for small fixed-size integers | Square and take middle digit |
| Folding hashing | Depends on number of digits | More digits mean more chunks to add |
| String hashing | `O(length of string)` | Each character must be processed |

Important idea:

> Hashing is usually average-case `O(1)`, but this depends on good distribution and a reasonable loading factor.

Worst case can become slow if many keys collide.

# Part D: Common Beginner Mistakes

## 26. Mistake 1: Thinking Hashing Is Always O(1)

Wrong idea:

```
Hashing is always O(1)
```

Correct idea:

```
Hashing is average-case O(1) when keys are distributed well.
```

If the hash function is bad, many keys collide and searching can become slow.

---

## 27. Mistake 2: Choosing a Table Size Without Thinking

A poor table size can increase collisions.

Example:

```
Table size = 10
Keys end in 5
```

Many keys may map to index `5`.

Better idea:

```
Use a prime table size such as 11, 13, or 17 when suitable.
```

---

## 28. Mistake 3: Using the Last Digit When Last Digits Are Similar

If all keys end in the same digit, using `% 10` is poor.

Example:

```
5, 35, 95, 145, 175
```

All end in `5`, so `% 10` sends them all to index `5`.

Better idea:

> Choose a function that uses a more useful part of the key.

---

## 29. Mistake 4: Forgetting Determinism

A hash function must return the same index for the same key every time.

Wrong:

```
H(25) = 5 today
H(25) = 8 tomorrow
```

Correct:

```
H(25) = 5 every time
```

---

## 30. Mistake 5: Thinking Collision Methods Fix Everything

Collision methods help, but they cannot fully fix a terrible hash function.

If every key goes to one place, even chaining becomes slow.

Simple lesson:

```
Good collision method + bad hash function = still bad performance
```

---

## 31. Mistake 6: Forgetting to Convert Strings to Numbers

Arrays use numeric indexes.

Strings must be converted to numbers first.

Example:

```
"ABC" → 65 + 66 + 67 = 198
198 % table_size = index
```

---

## 32. Mistake 7: Overfilling a Closed Hash Table

Closed hashing methods need empty spaces.

If the table is almost full:

```
Insertion becomes slower.
Search becomes slower.
Collisions become common.
Probe sequences become long.
```

For linear probing, remember:

```
lambda <= 0.5
```

---

# Part E: Final Hashing Workflow

---

## 33. Step-by-Step Checklist

When solving a hashing problem, ask these questions:

```
1. What type of keys do I have?
2. Do the keys have a pattern?
3. What table size should I choose?
4. Should the table size be prime?
5. Which hash function spreads the keys well?
6. What collision method should I use?
7. What is the loading factor?
8. How will insertion work?
9. How will search work?
10. How will deletion work?
```

## 34. Choosing a Method Quickly

| Situation | Good choice |
|---|---|
| Extra memory is allowed and deletion matters | Chaining |
| Simple array-only method with sparse table | Linear probing |
| Need less clustering than linear probing | Quadratic probing |
| Need stronger spreading and can design `H2(x)` | Double hashing |
| Integer keys with no bad pattern | Modulus hashing may be enough |
| Long numeric keys | Folding hashing may help |
| String keys | Convert characters to numbers first |

## 35. Very Important Formulas

### Loading Factor

```
lambda = n / size
```

### Modulus Hashing

```
H(x) = x % size
```

### Better Hash Function for Some Patterns

```
H(x) = (x / 10) % 10
```

### Linear Probing

```
H'(x) = (H(x) + i) % size
```

### Quadratic Probing

```
H'(x) = (H(x) + i²) % size
```

**Double Hashing**

```
H'(x) = (H1(x) + i * H2(x)) % size
```

**String ASCII Sum**

```
sum = ASCII(char1) + ASCII(char2) + ...
index = sum % size
```

---

# Part F: Mini Practice

---

## 36. Mini Practice 1: Modulus Hashing

Use table size `13`.

Find the index for key `28`.

```
28 % 13 = 2
```

Answer:

```
Index = 2
```

---

## 37. Mini Practice 2: Bad Hash Function

Keys:

```
10, 20, 30, 40
```

Hash function:

```
H(x) = x % 10
```

Calculations:

```
10 % 10 = 0
20 % 10 = 0
30 % 10 = 0
40 % 10 = 0
```

All keys go to index `0`.

Answer:

```
This is bad distribution.
```

---

## 38. Mini Practice 3: Mid-Square

Key:

```
14
```

Square:

```
14² = 196
```

Middle digit:

```
9
```

Answer:

```
Index = 9
```

If table size is smaller, apply modulus.

---

## 39. Mini Practice 4: Folding

Key:

```
987654
```

Break into pieces:

```
98, 76, 54
```

Add:

```
98 + 76 + 54 = 228
```

If table size is `13`:

```
228 % 13 = 7
```

Answer:

```
Index = 7
```

---

## 40. Mini Practice 5: String Hashing

String:

```
AB
```

ASCII values:

```
A = 65
B = 66
```

Sum:

```
65 + 66 = 131
```

If table size is `13`:

```
131 % 13 = 1
```

Answer:

```
"AB" maps to index 1
```

---

# Part G: Quick Revision Sheet

---

## 41. Phase 5 Quick Revision

| Concept | Simple meaning |
|---|---|
| Hash function | Converts a key into an index |
| Uniform distribution | Spreads keys evenly |
| Deterministic | Same key gives same index every time |
| Table size | Number of slots or buckets |
| Prime table size | Often reduces regular collision patterns |
| Loading factor | How full the table is |
| `lambda = n / size` | Loading factor formula |
| Modulus hashing | Uses remainder: `key % size` |
| Mid-square hashing | Square key, take middle digit(s) |
| Folding hashing | Break large key into parts and add them |
| String hashing | Convert characters to numbers first |
| Chaining | Store collisions in linked lists |
| Linear probing | Move one slot at a time |
| Quadratic probing | Use square jumps |
| Double hashing | Use a second hash function for jump size |

---

## 42. Phase 5 Full Summary

Phase 5 teaches how to design good hash functions and how to choose a hashing method.

A hash function converts a key into an index:

```
Key → Hash Function → Index
```

A good hash function should spread keys evenly across the table. This is called uniform distribution.

A good hash function must also be deterministic. This means the same key must always give the same index.

The programmer must understand the data before choosing a hash function. If all keys end in `5`, then `key % 10` is a poor choice because all keys go to index `5`.

Table size matters. For modulus hashing, prime table sizes like `11`, `13`, and `17` often help reduce regular collision patterns.

The loading factor is:

```
lambda = n / size
```

In chaining, `lambda` can be greater than `1` because chains can grow.

In linear probing, keep:

```
lambda <= 0.5
```

This keeps the table at most half full, leaving empty slots for probing.

The main hash functions in this phase are:

```
Modulus hashing
Mid-square hashing
Folding hashing
String hashing
```

Modulus hashing uses:

```
H(x) = x % size
```

Mid-square hashing squares the key and takes the middle digit.

Folding hashing breaks a large key into parts and adds them.

String hashing converts characters into numbers, such as ASCII values, before applying modulus.

To choose a collision method:

```
Use chaining when deletion is important and extra memory is allowed.
Use linear probing when you want a simple array-only method and the table is
sparse.
Use quadratic probing to reduce primary clustering.
Use double hashing when you want stronger spreading and can design a good second
hash function.
```

The final big idea is:

```
Good hashing = good hash function + good table size + suitable collision method
```

---

## 43. What You Should Be Able to Explain After Phase 5

After studying this phase, you should be able to explain:

1. What a hash function does.
2. Why uniform distribution matters.
3. What deterministic output means.
4. Why the same key must always give the same index.
5. Why the key pattern matters.
6. Why `key % 10` can be bad for keys ending in the same digit.
7. What loading factor means.
8. How to calculate `lambda = n / size`.
9. Why linear probing should keep `lambda <= 0.5`.
10. Why chaining can allow `lambda > 1`.
11. Why prime table sizes are useful.
12. How modulus hashing works.
13. How mid-square hashing works.
14. How folding hashing works.
15. How string hashing works using ASCII values.
16. What built-in hashing means.

17. Why built-in hashing still needs understanding of collisions.
18. When to choose chaining.
19. When to choose linear probing.
20. When to choose quadratic probing.
21. When to choose double hashing.
22. Why double hashing needs a good `H2(x)`.
23. Why hashing is average-case `O(1)`, not always guaranteed `O(1)`.
24. How to use the final hashing checklist.

---

## 44. Final Simple Explanation

Hashing tries to store and find data quickly by calculating where the key should go.

Earlier phases answered this question:

```
What do we do when collisions happen?
```

Phase 5 answers this question:

```
How do we choose a hash function so collisions are reduced?
```

A good hash function spreads keys evenly.

A bad hash function sends too many keys to the same place.

So the heart of Phase 5 is:

```
Choose a hash function that matches your data, choose a good table size, and
choose the right collision method.
```